CSC2103 Data Structures | Lecture 1

Algorithm Analysis, Arrays and Linked Lists

School of Computing (demo) | Week 1

Sample lecture deck compiled from Wikipedia (CC BY-SA 4.0) for a demo.
Not official material of any university. See LICENSE.md for sources and attribution.
Wikipedia articles used: Big O notation, Dynamic array, Linked list.

Learning outcomes

- Describe the growth of running time using Big O notation
- Rank common growth rates: $O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$, $O(2^n)$
- Explain why appending to a dynamic array is $O(1)$ amortised
- Compare arrays and linked lists and choose the right one for a task

Why analyse algorithms?

- Running time grows with the input size **n**
- We count basic operations, not seconds: the result does not depend on the machine
- **Worst case**: the most work for any input of size n (the usual guarantee)
- **Average case**: expected work over a distribution of inputs
- We care about behaviour for large n, so constants and small terms are ignored

Big O notation

- **Definition:** $f(n) = O(g(n))$ if there are constants $c > 0$ and n_0 such that $|f(n)| \leq c \cdot g(n)$ for all $n \geq n_0$
- Big O gives an **upper bound** on the growth rate of f
- Drop constant factors and lower-order terms:
  - $3n^2 + 5n + 2 = O(n^2)$
  - Proof: for $n \geq 1$, $3n^2 + 5n + 2 \leq 3n^2 + 5n^2 + 2n^2 = 10n^2$, so $c = 10$, $n_0 = 1$ works

Big Omega and Big Theta

Notation	Meaning	Informally
$f(n) = O(g(n))$	f grows no faster than g	upper bound
$f(n) = \Omega(g(n))$	f grows at least as fast as g	lower bound
$f(n) = \Theta(g(n))$	both $O(g(n))$ and $\Omega(g(n))$	tight bound

- $3n^2 + 5n + 2$ is $\Theta(n^2)$: it is both $O(n^2)$ and $\Omega(n^2)$
- In practice people often write O when they mean the tight bound Θ

Common growth rates

Order	Name	Example	n = 1,000
O(1)	constant	read a[i]	1
O(log n)	logarithmic	binary search in a sorted array	≈ 10
O(n)	linear	linear search	1,000
O(n log n)	linearithmic	merge sort, heapsort	≈ 10,000
$O(n^2)$	quadratic	two nested loops over the input	1,000,000
$O(2^n)$	exponential	try every subset	$\approx 10^{301}$

Analysing loops

```
// Nested loops: n * n iterations -> O(n^2)
for (let i = 0; i < n; i++)
  for (let j = 0; j < n; j++)
    sum += a[i] * a[j];          // O(1) work

// Doubling counter: about log2(n) iterations -> O(log n)
for (let i = 1; i < n; i *= 2)
  steps++;
```

- Sequential blocks add: $O(n) + O(n^2) = O(n^2)$
- Nested loops multiply their iteration counts

Arrays (static)

- Elements stored in **one contiguous block** of memory
- Address of element i = base address + i × element size, so **access by index is O(1)**
- Capacity is fixed when the array is created
- Inserting or deleting in the middle shifts the elements after it: **O(n)**
- Good cache locality: neighbours sit next to each other in memory

Dynamic arrays

- A variable-size list built on a fixed-size array plus a **capacity**
- Elements 0 .. size−1 are used; the rest of the capacity is reserved for growth
- When size = capacity, allocate a **bigger array** (for example double the capacity) and copy every element across
- Examples: Java `ArrayList`, C++ `std::vector`, Python `list`, JavaScript arrays
- Common growth factors are between 1.5 and 2

Append with geometric growth

```
function append(arr, x) {
  if (arr.size === arr.capacity) {              // full: grow
    const bigger = new Array(arr.capacity * 2);
    for (let i = 0; i < arr.size; i++)
      bigger[i] = arr.data[i];                  // copy: O(n)
    arr.data = bigger;
    arr.capacity *= 2;
  }
  arr.data[arr.size++] = x;                     // usual case: O(1)
}
```

Amortised analysis of append

- One append can cost O(n) (when it triggers a copy), but copies are rare
- With doubling, n appends copy at most 1 + 2 + 4 + … + n/2 < n elements in total
- Total work for n appends < n writes + n copies = **O(n)**, so each append is **O(1) amortised**
- Growing by a constant amount (for example +10 slots) instead gives O(n) per append on average and $O(n^2)$ in total
- Trade-off: a larger growth factor means fewer copies but more unused capacity

Linked lists

- A sequence of **nodes**; each node stores a value and a reference (**next**) to the next node
- The list is reached through its **head**; the last node's next is null
- Variants: singly linked, doubly linked (next and prev), circular
- Insert or delete at a known node: **O(1)**, no shifting
- Access the k-th element: **O(n)**, must walk from the head
- Costs: extra memory for references, poor cache locality

Linked list in JavaScript

```
class Node {
  constructor(value, next = null) { this.value = value; this.next = next; }
}

function pushFront(list, value) {        // O(1)
  list.head = new Node(value, list.head);
}

function find(list, value) {             // O(n)
  for (let cur = list.head; cur !== null; cur = cur.next)
    if (cur.value === value) return cur;
  return null;
}
```

Linked list vs dynamic array

Operation	Linked list	Dynamic array
Access by index	$\Theta(n)$	$\Theta(1)$
Insert/delete at beginning	$\Theta(1)$	$\Theta(n)$
Insert/delete at end	$\Theta(1)$ when the last node is known, $\Theta(n)$ otherwise	$\Theta(1)$ amortised
Insert/delete in middle	search time + $\Theta(1)$	$\Theta(n)$
Wasted space	$\Theta(n)$ (references)	$\Theta(n)$ (spare capacity)

Deleting the last node of a singly linked list still needs the node before it, so it is $\Theta(n)$ without a doubly linked list.

Summary and tutorial

- Big O describes growth for large n; drop constants and lower-order terms
- Dynamic arrays: O(1) index access, O(1) amortised append, O(n) insert in the middle
- Linked lists: O(1) insert/delete at a known node, O(n) access by position

Tutorial 1: (1) Show that $5n^2 + 3n \log n = O(n^2)$ by giving c and n_0. (2) If a dynamic array triples its capacity instead of doubling, is append still O(1) amortised? Why? (3) Write a function that returns the middle node of a singly linked list in one pass.

References and licence

- Wikipedia, *Big O notation*, https://en.wikipedia.org/wiki/Big_O_notation (accessed 10 Oct 2026)
- Wikipedia, *Dynamic array*, https://en.wikipedia.org/wiki/Dynamic_array (accessed 10 Oct 2026)
- Wikipedia, *Linked list*, https://en.wikipedia.org/wiki/Linked_list (accessed 10 Oct 2026)

Text adapted and condensed from the articles above, licensed CC BY-SA 4.0 (https://creativecommons.org/licenses/by-sa/4.0/). This deck is shared under the same licence. Code snippets written for this deck.